UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
PROFESSOR: FÁBIO LUIZ LEITE JÚNIOR

MARIA EDUARDA DA NÓBREGA

ADRIELLY CARLA FERREIRA DE MELO

LINK DO REPOSITÓRIO: https://github.com/eduardanb/sudoku-solver-taal.git

PROJETO DE ANÁLISE DE ALGORITMOS: SUDOKU SOLVER

CAMPINA GRANDE - PB

2026

SUMÁRIO

1. INTRODUÇÃO **3**
1.1. Contextualização do Problema 3
1.2. Objetivos 4
1.2.1. Gerais 4
1.2.2. Específicos 4
2. METODOLOGIA E IMPLEMENTAÇÃO **5**
2.1. Estrutura do Sistema Desenvolvido 5
2.1.1. Pacote experiment 5
2.1.2. Pacote io 6
2.1.3. Pacote model 6
2.1.4. Pacote solver e subpacotes 7
2.1.5. Pacotes heuristics e utils 8
2.1.6. Pacote metrics 9
2.2. Formato de Dados de Entrada e Saída 9
2.2.1. Arquivos de entrada 9
2.2.2. Saída em console 10
2.2.3. Arquivos de saída 11
2.3. Métricas e Avaliação 12
2.3.1. Métricas de desempenho coletadas 12
2.3.2. Protocolo de experimentos em lote 13
2.3.3. Variação da corretude das soluções 14
3. ALGORITMOS IMPLEMENTADOS **14**
3.1. Backtracking 15
3.1.1. Análise de Corretude 15
3.1.2. Análise de Desempenho 16
3.1.2.1. Espacial 16
3.1.2.2. Temporal 17
3.2. Branch and Bound 18
3.2.1. Análise de Corretude 20
3.2.2. Análise de Desempenho 21
3.2.2.1. Espacial 21
3.2.2.2. Temporal 22
4. EXPERIMENTOS E RESULTADOS **23**
4.1. Instâncias de Teste 23
4.2. Resultados Obtidos 24
4.3. Síntese dos Achados Experimentais 28
5. ANÁLISE E DISCUSSÃO **29**
5.1. Comparação das Estratégias 29
6. CONCLUSÃO **31**
7. REFERÊNCIAS **32**

1. INTRODUÇÃO

Durante a disciplina de Técnicas de Análise de Algoritmos, foram propostas atividades em equipe com o objetivo de desenvolver soluções computacionais aplicando técnicas algorítmicas estudadas em sala de aula. Cada grupo deveria selecionar um dos temas sugeridos pelo professor e implementar uma solução baseada nos métodos de projeto de algoritmos abordados ao longo da disciplina.

O tema escolhido por nossa equipe foi o Sudoku Solver, correspondente ao tema 13 da lista de propostas. O problema consiste em desenvolver um sistema capaz de resolver automaticamente um tabuleiro de Sudoku, utilizando estratégias algorítmicas adequadas para encontrar uma solução válida.

Para a resolução do problema foram consideradas diferentes técnicas de projeto de algoritmos, entre elas Backtracking, Branch and Bound, Greedy e Dynamic Programming, permitindo a análise de suas aplicações e desempenho no contexto da resolução do Sudoku.

1.1. Contextualização do Problema

O Sudoku é um jogo de lógica antigo, amplamente conhecido que consiste em preencher um tabuleiro de 9x9 células, dividido em nove sub-grades de 3x3, com números de 1 a 9. O objetivo do jogo é completar o tabuleiro respeitando determinadas restrições: cada número deve aparecer apenas uma vez em cada linha, em cada coluna e em cada sub-grade 3x3.

No contexto computacional, o Sudoku pode ser tratado como um problema de busca com restrições, no qual um algoritmo deve encontrar valores válidos para as células vazias de um tabuleiro parcialmente preenchido. Nesse problema, algumas posições já possuem números definidos, enquanto outras estão vazias e precisam ser completadas de forma que todas as regras do jogo sejam satisfeitas.

No problema abordado neste projeto, o tabuleiro de entrada é representado por uma matriz 9x9 de números inteiros, onde o valor 0 representa uma célula vazia que precisa ser preenchida. O algoritmo desenvolvido deve analisar o tabuleiro inicial e preencher os espaços vazios com números de 1 a 9 de forma que nenhuma das restrições do Sudoku seja violada.

A entrada do programa consiste em 9 linhas contendo 9 números inteiros cada, representando o estado inicial do tabuleiro. O algoritmo deve então produzir como saída o tabuleiro completo solucionado, sem quebrar nenhuma regra do jogo.

Do ponto de vista da análise de algoritmos, o problema de resolver Sudoku é interessante por envolver espaços de busca potencialmente grandes, o que torna necessária a utilização de técnicas algorítmicas eficientes para explorar as possíveis combinações de preenchimento do tabuleiro. Dessa forma, métodos de projeto de algoritmos como Backtracking, Branch and Bound, Greedy e Dynamic Programming podem ser investigados e comparados quanto ao seu desempenho na resolução do problema.

Além da obtenção de uma solução válida, também é possível analisar métricas importantes relacionadas à execução do algoritmo, como tempo de resolução, número de nós visitados durante a busca e tamanho da árvore de busca gerada, permitindo avaliar a eficiência das estratégias utilizadas.

1.2. Objetivos

Nesta seção, são apresentados os objetivos gerais e específicos da presente atividade.

1.2.1. Gerais

O objetivo geral do presente trabalho é desenvolver um algoritmo capaz de resolver automaticamente um tabuleiro de Sudoku 9x9 parcialmente preenchido, entre os níveis fácil, médio e difícil, aplicando técnicas de projeto e análise de algoritmos estudados na disciplina.

1.2.2. Específicos

Os objetivos específicos focam entre implementar um algoritmo capaz de preencher corretamente as células vazias de um tabuleiro de Sudoku, respeitando todas as regras do jogo. Para isso, são utilizadas diferentes técnicas de projeto de algoritmos, como Backtracking, Branch and Bound, Greedy e Dynamic Programming, com o intuito de investigar suas aplicações na resolução do problema.

Além da implementação das abordagens propostas, busca-se avaliar o desempenho de cada técnica por meio de métricas como tempo de execução,

número de nós visitados e tamanho da árvore de busca gerada durante o processo de resolução. Esses indicadores permitem analisar o comportamento dos algoritmos e compreender sua eficiência computacional.

Também são comparados os resultados obtidos a partir de diferentes instâncias do problema, considerando tabuleiros classificados como fáceis, médios e difíceis. Essa comparação possibilita observar como cada técnica se comporta diante de diferentes níveis de complexidade do Sudoku.

Por fim, é realizada uma análise das técnicas algorítmicas utilizadas, identificando suas principais vantagens e limitações na resolução do problema, bem como discutindo sua adequação e desempenho para esse tipo de aplicação.

2. METODOLOGIA E IMPLEMENTAÇÃO

Nesta seção, são descritos a estrutura do sistema desenvolvido, o formato dos dados de entrada e saída, bem como as métricas utilizadas para avaliação.

2.1. Estrutura do Sistema Desenvolvido

O sistema foi projetado seguindo o princípio de separação de responsabilidades, distribuindo as funcionalidades em pacotes coesos e com acoplamento reduzido. A estrutura de diretórios segue a convenção Maven, com código-fonte em *src/main/java*, recursos estáticos em *src/main/resources* e testes automatizados em *src/test/java*. A organização interna do código é distribuída em seis pacotes principais, cada um com responsabilidade bem delimitada, conforme descrito a seguir.

2.1.1. Pacote experiment

O pacote br.edu.sudoku.experiment concentra o ponto de entrada interativo do sistema, representado pela classe ExperimentRunner. Esta classe implementa um laço de interação via terminal que permite ao usuário selecionar, a cada execução, o algoritmo desejado (Backtracking, Branch and Bound, Greedy ou Programação Dinâmica) e o nível de dificuldade do jogo (fácil, médio ou difícil). Internamente, o ExperimentRunner é responsável por orquestrar o fluxo completo de uma execução: carregamento do tabuleiro, instanciação do solver, temporização da

resolução e exibição dos resultados no console, incluindo o tabuleiro inicial, o tabuleiro resolvido e as métricas de desempenho coletadas.

Além do ExperimentRunner, o pacote também abriga a classe PerformanceComparisonTest, voltada à execução em lote: ela percorre automaticamente todas as combinações de algoritmo e dificuldade, repete cada experimento dez vezes e calcula médias e desvios padrão, provendo uma base estatística robusta para a comparação de desempenho.

2.1.2. Pacote io

O pacote br.edu.sudoku.io encapsula toda a lógica de entrada e saída do sistema. A classe SudokuReader é responsável por carregar um tabuleiro a partir de um arquivo de texto simples localizado no classpath, utilizando ClassLoader.getResourceAsStream para garantir compatibilidade com a estrutura de recursos do Maven. O arquivo é lido linha a linha com BufferedReader.readLine(), os valores são separados por espaço e o resultado é encapsulado em um objeto SudokuBoard.

A classe SudokuWriter oferece dois métodos: printBoard, que delega a exibição formatada ao próprio modelo, e writeToFile, que persiste a solução em um arquivo de texto com o mesmo formato da entrada. O nome do arquivo de saída é composto dinamicamente a partir do nível de dificuldade e do algoritmo utilizado, por exemplo, sudoku_medio_backtracking.txt, favorecendo a rastreabilidade dos resultados.

2.1.3. Pacote model

O pacote br.edu.sudoku.model contém a classe SudokuBoard, que representa o estado do tabuleiro em um dado momento. Internamente, o tabuleiro é mantido como uma matriz bidimensional int[9][9], onde o valor 0 indica uma célula ainda não preenchida. Adicionalmente, a classe mantém uma matriz booleana boolean[9][9] que registra, no momento da criação do objeto, quais células possuíam valores originalmente fixos (as pistas do jogo). Essa distinção é utilizada na impressão do tabuleiro: células fixas são exibidas em azul ciano via código de escape ANSI

(\u001B[36m), facilitando a diferenciação visual entre as pistas fornecidas e os valores inseridos pelo solver durante a resolução.

A separação entre células fixas e células preenchidas pelo solver é especialmente importante para a integridade da busca: os algoritmos nunca tentam sobrescrever células fixas, pois operam apenas sobre posições onde board.get(linha, coluna) == 0.

2.1.4. Pacote solver e subpacotes

O pacote br.edu.sudoku.solver define a interface central SudokuSolver, que declara o contrato único compartilhado por todos os algoritmos: o método boolean solve(SudokuBoard tabuleiro, Metrics metricas). Esse contrato permite que o ExperimentRunner trabalhe de forma polimórfica com qualquer implementação de solver, sem precisar conhecer os detalhes internos de cada algoritmo. Os algoritmos são organizados em quatro subpacotes independentes, cada um contendo uma interface específica e sua respectiva classe de implementação. A tabela a seguir resume os quatro algoritmos propostos:

Algorimo	Estratégia de seleção	Heurística	Status
Backtrakcing	Varredura linear (linha x coluna)	Nenhuma	Implementado
Branch and Bound	MRV - Célula com menos candidatos	MRVHeuristic	Implementado
Greedy	-	-	Próxima entrega
Prog. Dinâmica	-	-	Próxima entrega

Tabela 1 - Algoritmos de resolução previstos no sistema.

O Backtracking (BacktrackingSolver) implementa a busca por retrocesso clássica: percorre as células vazias em ordem linear (linha por linha, coluna por coluna), tenta inserir os valores de 1 a 9 em cada posição e retrocede ao detectar uma violação de restrição. A cada tentativa, o nó visitado é contabilizado pela instância de Metrics; cada retrocesso incrementa o contador de backtracks. O algoritmo suporta, opcionalmente, um modo de visualização passo a passo, ativado

pelo parâmetro booleano visualizar no construtor, que exibe o estado do tabuleiro a cada atribuição com um atraso de 150 ms, tornando o processo de busca observável em tempo real.

O Branch and Bound (BranchAndBoundSolver) incorpora a heurística MRV (Minimum Remaining Values) para guiar a seleção de variáveis. Em vez de escolher a próxima célula vazia em ordem linear, o algoritmo consulta a classe MRVHeuristic, que varre todas as células vazias e retorna aquela com o menor número de valores candidatos válidos. Ao priorizar as células mais restritas, a heurística reduz significativamente o espaço de busca explorado, pois detecta conflitos mais cedo na árvore de decisão. Como otimização adicional, quando uma célula com apenas um candidato é encontrada, o método retorna imediatamente sem varrer o restante do tabuleiro. Os algoritmos Greedy e Programação Dinâmica possuem estrutura de classe e interface criadas, mas a lógica de resolução ainda não foi implementada, estando previstos para a segunda entrega do sistema.

2.1.5. Pacotes heuristics e utils

O pacote br.edu.sudoku.heuristics define a interface VariableOrderingHeuristic, com o método int[] selecionarCelula(SudokuBoard tabuleiro), e a implementação MRVHeuristic. O método retorna um array de dois elementos com as coordenadas [linha, coluna] da célula escolhida. Internamente, para cada célula vazia, o método auxiliar obterCandidatos constrói um array de até 10 posições: a posição de índice 0 armazena a contagem de candidatos válidos, e as posições 1 a 9 armazenam os próprios valores candidatos. Essa estrutura é reutilizada tanto na heurística quanto no BranchAndBoundSolver, evitando alocações redundantes.

O pacote br.edu.sudoku.utils contém a classe SudokuValidator, cujo método estático isValid(SudokuBoard board, int linha, int col, int num) verifica, em tempo O(n), se a inserção de um valor em uma célula viola alguma das três restrições do Sudoku: unicidade na linha, unicidade na coluna e unicidade na subgrade 3×3. A subgrade relevante é determinada pelas fórmulas (linha / 3) * 3 e (col / 3) * 3, que mapeiam qualquer célula para o canto superior esquerdo de seu bloco. Valores fora

do intervalo [1, 9] são imediatamente rejeitados, antes mesmo de qualquer verificação estrutural.

2.1.6. Pacote metrics

O pacote br.edu.sudoku.metrics encapsula a coleta de dados de desempenho por meio da classe Metrics. Cada execução de solver recebe uma instância isolada, garantindo que os contadores não sejam compartilhados entre experimentos distintos. A classe expõe dois contadores incrementais, visitedNodes e backtracks, ambos do tipo long para suportar instâncias de alta dificuldade que podem gerar volumes elevados de chamadas recursivas sem risco de overflow.

2.2. Formato de Dados de Entrada e Saída

2.2.1. Arquivos de entrada

Os tabuleiros de entrada são armazenados como arquivos de texto simples (.txt) no diretório *src/main/resources/sudokus/*. Cada arquivo representa um tabuleiro 9×9 cujas linhas correspondem diretamente às linhas do jogo: nove valores numéricos inteiros separados por espaço simples. Células sem valor predefinido são representadas pelo inteiro 0. O sistema disponibiliza três instâncias com graus de dificuldade distintos:

- sudoku_facil.txt: jogo com elevado número de pistas, exigindo menor exploração.
- sudoku_medio.txt: jogo com número intermediário de pistas.
- sudoku_difícil.txt: jogo com poucas pistas, exigindo uma busca mais extensa.

A seguir, apresenta-se um exemplo do formato de entrada:

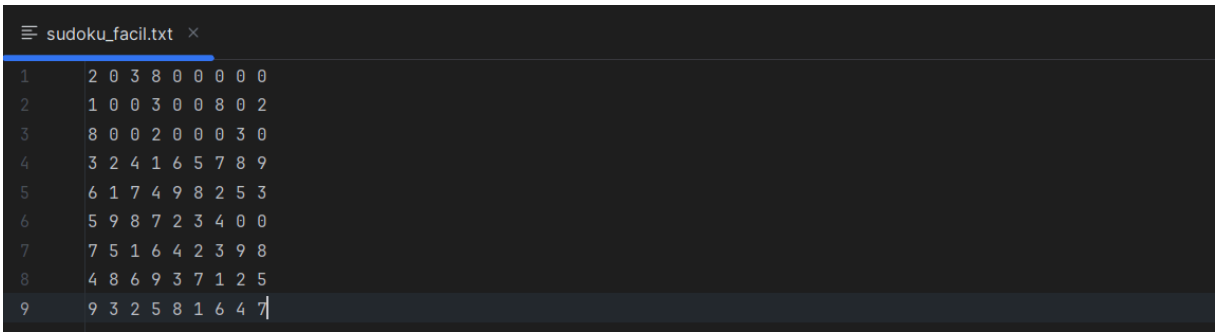

Figura 1 - Exemplo de arquivo de entrada, nível fácil.

Internamente, a leitura é realizada pela classe SudokuReader, que utiliza ClassLoader.getResourceAsStream(path) para localizar o arquivo no classpath, percorre as nove linhas com BufferedReader.readLine() e converte cada token em inteiro via Integer.parseInt. O resultado é encapsulado em um objeto SudokuBoard, que registra automaticamente quais posições eram não-nulas como células fixas.

2.2.2. Saída em console

Durante a execução interativa, o sistema exibe no console o tabuleiro inicial e, ao término da resolução, o tabuleiro completo, por meio do método SudokuBoard.printBoard(). A formatação adota separadores verticais e horizontais para demarcar os nove blocos 3×3 do jogo. Células cujos valores fazem parte das pistas originais são exibidas em azul ciano (código ANSI \u001B[36m), enquanto os valores inseridos pelo solver são exibidos com a cor padrão do terminal. Ao término da resolução, são apresentadas as métricas de desempenho: tempo de execução em milissegundos, total de nós visitados e número de backtracks.

A seguir, apresenta-se uma amostra do formato da saída no console:

Figura 2 - Início de execução.

```
=== Sudoku Solver (Branch and Bound) ===
Dificuldade: DIFICIL
Passo: 66
Tentando colocar 9 em (6,7)

+-------+-------+-------+
| 4 7 9 | 8 6 5 | 2 3 1 |
| 5 2 6 | 3 7 1 | 8 4 9 |
| 8 3 1 | 4 2 9 | 5 6 7 |
+-------+-------+-------+
| 6 9 8 | 5 3 7 | 1 2 4 |
| 7 4 2 | 6 1 8 | 9 5 3 |
| 1 5 3 | 2 9 4 | 7 8 6 |
+-------+-------+-------+
| 3 1 4 | 7 5 2 | 6 9 8 |
| 9 6 5 | 1 8 3 | 4 7 2 |
| 2 8 7 | 9 4 6 | 3 1 5 |
+-------+-------+-------+
```

Figura 3 - Exemplo de finalização de Sudoku, técnica Branch and Bound, nível difícil.

```
Sudoku resolvido:

+-------+-------+-------+
| 4 7 9 | 8 6 5 | 2 3 1 |
| 5 2 6 | 3 7 1 | 8 4 9 |
| 8 3 1 | 4 2 9 | 5 6 7 |
+-------+-------+-------+
| 6 9 8 | 5 3 7 | 1 2 4 |
| 7 4 2 | 6 1 8 | 9 5 3 |
| 1 5 3 | 2 9 4 | 7 8 6 |
+-------+-------+-------+
| 3 1 4 | 7 5 2 | 6 9 8 |
| 9 6 5 | 1 8 3 | 4 7 2 |
| 2 8 7 | 9 4 6 | 3 1 5 |
+-------+-------+-------+

================ RESULTADOS ================
Tempo de execução: 10381 ms
Nós visitados: 53
Backtracks: 14
============================================
```

Figura 4 - Amostra do sudoku resolvido no terminal com as métricas.

2.2.3. Arquivos de saída

Após a resolução bem-sucedida, o tabuleiro completo é gravado em arquivo de texto no mesmo diretório de recursos, com nome composto dinamicamente no padrão sudoku_<dificuldade>_<algoritmo>.txt, por exemplo, sudoku_medio_backtracking.txt. O formato de saída é idêntico ao de entrada: nove linhas de nove valores separados por espaço. Essa decisão garante interoperabilidade total entre os arquivos do sistema: qualquer saída pode ser reutilizada como entrada sem nenhuma conversão adicional, facilitando a validação manual das soluções e a comparação direta entre os resultados de diferentes algoritmos sobre o mesmo tabuleiro.

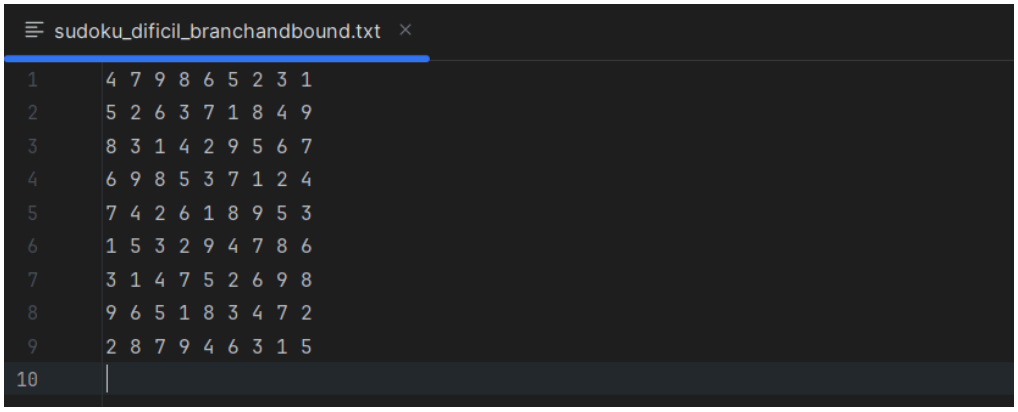

Figura 5 - Saída em arquivo .txt do Sudoku nível difícil utilizando Branch and Bound.

2.3. Métricas e Avaliação

A avaliação comparativa dos algoritmos foi estruturada em três dimensões complementares: eficiência de busca, custo temporal e corretude da solução.

2.3.1. Métricas de desempenho coletadas

Todas as métricas de busca são coletadas durante a execução por meio da classe Metrics, cuja instância é passada por parâmetro a cada chamada do método solve. Isso garante isolamento total dos contadores por execução, sem efeitos colaterais entre experimentos.

Métrica	Unidade	Coletada
Nós visitados	Contagem inteira	Metrics.getVisitedNodes()
Backtracks	Contagem inteira	Metrics.getBacktracks()
Tempo de execução	Milissegundos (ms)	System.nanoTime()
Desvio padrão	Milissegundos (ms)	PerformanceComparison Test

Tabela 2 - Métricas coletadas pelo sistema.

O número de nós visitados é registrado a cada chamada recursiva ao método de resolução, seja ele backtracking ou branch and bound, por meio da invocação do método Metrics.incrementVisitedNodes() antes de qualquer outra operação. Esse contador reflete o tamanho efetivo do espaço de busca explorado, abrangendo tanto

os nós que resultaram em atribuições parciais válidas quanto aqueles que levaram a conflitos imediatos. Dessa forma, constitui uma métrica representativa da complexidade computacional intrínseca de cada algoritmo, independente do hardware utilizado.

O número de backtracks ocorre sempre que uma tentativa de atribuição falha, ou seja, quando nenhum valor válido pode ser inserido na célula atual. Nesses casos, o algoritmo desfaz a última atribuição, restaura a célula para zero e invoca Metrics.incrementBacktracks(). Essa métrica indica a frequência com que o algoritmo se depara com becos sem saída na árvore de busca. Valores elevados sugerem que muitas atribuições inconsistentes foram realizadas antes de serem detectadas, evidenciando ineficiência na estratégia de seleção de variáveis.

O tempo de execução é medido utilizando System.nanoTime() imediatamente antes e após a chamada a solver.solve(), sendo o intervalo convertido para milissegundos com precisão de três casas decimais. A utilização de nanoTime, em vez de currentTimeMillis, garante maior resolução temporal e independência de ajustes de relógio do sistema operacional, permitindo uma avaliação mais precisa do desempenho dos algoritmos implementados.

2.3.2. Protocolo de experimentos em lote

Para reduzir o impacto de variações do ambiente de execução, como aquecimento da JVM, coleta de lixo e escalonamento de threads, os experimentos comparativos são conduzidos pela classe PerformanceComparisonTest, seguindo o protocolo abaixo:

- Cada combinação de algoritmo e nível de dificuldade é executada dez vezes consecutivas, utilizando a constante EXECUCOES = 10.
- Para cada execução, o tabuleiro é recarregado a partir do arquivo, garantindo estado inicial idêntico entre as repetições.
- Uma nova instância de Metrics é criada a cada execução, evitando o acúmulo de contagens.
- Ao final, são calculadas a média aritmética e o desvio padrão do tempo de execução, a média de nós visitados e a média de backtracks.

O desvio padrão do tempo de execução funciona como indicador de estabilidade do algoritmo. Valores baixos indicam comportamento consistente entre execuções, enquanto valores altos sugerem sensibilidade a fatores externos, como coleta de lixo ou variações de carga da JVM.

2.3.3. Variação da corretude das soluções

A validade das soluções produzidas por cada algoritmo é verificada automaticamente pelos testes JUnit 5 implementados em BacktrackingSolverTest, BranchAndBoundSolverTest e SolverComparisonTest. O método auxiliar tabuleiroCompletoEValido percorre todas as 81 células do tabuleiro solução e, para cada uma, verifica duas condições: (1) o valor atribuído está no intervalo [1, 9]; e (2) esse valor é consistente com as restrições do jogo, utilizando SudokuValidator.isValid após temporariamente zerar a célula para evitar falso conflito da célula consigo mesma.

Os testes unitários em SudokuValidatorTest cobrem individualmente os seis cenários relevantes de validação: inserção válida, conflito de linha, conflito de coluna, conflito de subgrade 3×3, valor zero e valor acima de 9. Essa cobertura garante que a função de validação, base de todos os solvers, esteja correta antes que qualquer algoritmo seja avaliado. Os níveis de dificuldade e os algoritmos a serem testados são configurados dinamicamente por meio de System.setProperty, permitindo selecionar qualquer combinação sem necessidade de recompilação.

3. ALGORITMOS IMPLEMENTADOS

Esta seção apresenta uma análise detalhada dos algoritmos de resolução implementados no sistema, abordando seus fundamentos teóricos, decisões de implementação, corretude formal e complexidade computacional. Para cada algoritmo, são examinadas separadamente a complexidade espacial e a complexidade temporal, considerando tanto o caso geral quanto o comportamento esperado no contexto específico do Sudoku 9×9.

3.1. Backtracking

O Backtracking é uma técnica algorítmica de busca exaustiva com poda, amplamente utilizada na resolução de problemas de satisfação de restrições. A ideia central consiste em construir a solução incrementalmente, atribuindo valores a variáveis uma a uma e abandonando imediatamente qualquer atribuição parcial que viole alguma restrição, sem precisar explorar todas as extensões possíveis daquela atribuição. No contexto do Sudoku, cada célula vazia corresponde a uma variável, e as restrições são as regras de unicidade por linha, coluna e subgrade 3×3.

A implementação do sistema segue esse paradigma de forma direta. A classe BacktrackingSolver percorre as células vazias em ordem linear, linha por linha da esquerda para a direita, tentando inserir os valores de 1 a 9 em cada posição. Quando nenhum valor é válido para a célula atual, a última atribuição é desfeita e o algoritmo retrocede para a célula anterior. Esse processo se repete recursivamente até que todas as células estejam preenchidas, indicando que a solução foi encontrada, ou até que o espaço de busca se esgote completamente, caracterizando um jogo sem solução.

Um aspecto relevante da implementação é o suporte ao modo de visualização passo a passo, ativado pelo parâmetro booleano visualizar no construtor. Quando ativo, o algoritmo exibe o estado do tabuleiro a cada atribuição e a cada retrocesso, com um atraso de 150 ms entre os passos por meio de Thread.sleep. Esse recurso é desativado nos testes automatizados e nos experimentos em lote, onde a instância é criada com new BacktrackingSolver(false), eliminando o overhead de I/O e de temporização do ciclo de medição. O nome do nível de dificuldade é recuperado dinamicamente por meio de System.getProperty("difficulty"), permitindo que tanto o ExperimentRunner quanto os testes JUnit configurem o rótulo exibido sem a necessidade de repassar o parâmetro pela cadeia de chamadas recursivas.

3.1.1. Análise de Corretude

A corretude de um algoritmo de busca com retrocesso pode ser demonstrada por três propriedades complementares: completude, validade e terminação.

O algoritmo é completo para instâncias de Sudoku bem formadas, pois o espaço de busca do Sudoku 9×9 é finito e limitado, com no máximo 9^{81} estados possíveis, embora a grande maioria seja eliminada pelas restrições logo nas primeiras atribuições. A cada nível de recursão, o algoritmo experimenta todos os valores candidatos para a célula atual, de 1 a 9, antes de retornar false. Se alguma combinação de valores leva a uma solução, ela será necessariamente alcançada, uma vez que o algoritmo explora sistematicamente todas as ramificações válidas da árvore de busca sem saltar nenhum nó. A condição de parada, que consiste em retornar true quando não existem mais células com valor 0, garante que a solução só é declarada quando o tabuleiro está integralmente preenchido.

A correção do laço principal pode ser estabelecida por meio de uma invariante de laço: ao início de cada iteração sobre os valores candidatos de uma célula, o tabuleiro encontra-se em um estado parcialmente preenchido que é consistente com todas as restrições do Sudoku, ou seja, nenhuma das atribuições realizadas até aquele ponto viola unicidade de linha, coluna ou subgrade. Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada com pistas que não se contradizem entre si. É mantida a cada iteração, pois a atribuição de um novo valor à célula corrente só ocorre após a confirmação de SudokuValidator.isValid, que garante que o valor não cria nenhum conflito com o estado atual; caso nenhum valor satisfaça a verificação, a célula permanece inalterada e o algoritmo retrocede, restaurando o estado consistente anterior. É satisfeita na terminação, pois quando o laço encerra com retorno true, todas as células foram preenchidas e a invariante de consistência global do tabuleiro está integralmente preservada.

Além disso, toda atribuição realizada pelo algoritmo é precedida pela verificação SudokuValidator.isValid(tabuleiro, linha, coluna, numero), que confirma que o valor candidato não viola nenhuma das três restrições simultaneamente: unicidade na linha, unicidade na coluna e unicidade na subgrade 3×3. Como essa verificação ocorre antes de cada atribuição, qualquer estado parcial mantido durante a execução permanece consistente com as regras do jogo. Consequentemente, quando o algoritmo retorna true, o tabuleiro está completo e cada uma das 81 células satisfaz todas as restrições. Essa propriedade é formalmente validada pelos testes em BacktrackingSolverTest, que percorrem todas as células da solução

retornada e invocam SudokuValidator.isValid célula a célula, após temporariamente zerá-las para evitar falso conflito da célula consigo mesma.

O algoritmo termina em tempo finito, uma vez que o número de células é estritamente limitado a 81, cada célula aceita no máximo nove valores distintos, e o mecanismo de retrocesso garante que nenhuma ramificação seja visitada mais de uma vez. A profundidade máxima da pilha de recursão corresponde ao número de células vazias no tabuleiro inicial, que é sempre menor que 81, impedindo qualquer possibilidade de recursão infinita.

3.1.2. Análise de Desempenho

3.1.2.1. Espacial

A complexidade espacial do Backtracking é composta por dois componentes principais: o espaço ocupado pela representação do tabuleiro e o espaço consumido pela pilha de chamadas recursivas. O tabuleiro é representado pela classe SudokuBoard como uma matriz int[9][9], que ocupa espaço constante independentemente da instância resolvida, uma vez que as dimensões do Sudoku são fixas em 9×9. A matriz booleana auxiliar boolean[9][9], responsável por registrar as células fixas, acrescenta outro bloco de tamanho constante. O espaço total dedicado ao modelo de dados é, portanto, O(1), ou seja, 81 inteiros de 4 bytes cada (324 bytes) mais 81 booleanos de 1 byte cada (81 bytes).

O componente dominante na análise espacial é a pilha de recursão. Cada chamada recursiva a backtrack empilha um novo frame de ativação contendo as variáveis locais de controle dos laços — linha, coluna e número — além do endereço de retorno e da referência ao objeto Metrics. A profundidade máxima da pilha é igual ao número de células vazias no tabuleiro inicial, denotado por n, que varia de zero, no caso de um tabuleiro já resolvido, até aproximadamente 64 nas instâncias mais difíceis do Sudoku padrão. A complexidade espacial total é, portanto, O(n), com $n \leq 81$.

Um aspecto crítico para essa complexidade favorável é o fato de o algoritmo realizar todas as atribuições e desfazimentos in-place na mesma instância de SudokuBoard, sem criar cópias do tabuleiro a cada nível de recursão. Essa decisão

de implementação é fundamental, pois se uma cópia fosse criada a cada chamada recursiva, o consumo de memória escalaria para O(n × 81), aumentando em cerca de duas ordens de grandeza o uso de memória nas instâncias mais difíceis.

3.1.2.2. Temporal

A análise da complexidade temporal do Backtracking para o Sudoku exige distinguir entre o pior caso teórico e o comportamento esperado na prática, considerando o papel fundamental das restrições na poda do espaço de busca. No pior caso teórico, o algoritmo explora todas as combinações possíveis de valores para as n células vazias antes de encontrar uma solução ou declarar insolubilidade. Com cada célula podendo receber até nove valores, o pior caso é $O(9^n)$. Para n = 81, isso resulta em $O(9^{81})$, um número da ordem de 10^{77}, computacionalmente impraticável. Na prática, contudo, esse limite superior é muito pessimista e nunca se materializa, pois as verificações de restrição eliminam a imensa maioria dos ramos logo nas primeiras atribuições, e a estrutura do jogo impõe uma densidade de restrições suficiente para tornar a busca tratável.

O mecanismo de poda reduz drasticamente o número de nós efetivamente explorados. Cada chamada a SudokuValidator.isValid tem custo O(1), pois percorre no máximo 27 células — nove da linha, nove da coluna e nove da subgrade 3×3 — e a maioria dos valores candidatos é descartada antes mesmo de gerar uma chamada recursiva. O número real de nós visitados depende fortemente da instância: jogos com mais pistas impõem mais restrições e reduzem o espaço de busca de forma mais agressiva. Para as três instâncias utilizadas neste trabalho — fácil, médio e difícil —, o número empírico de nós visitados é coletado pela classe Metrics e comparado diretamente com o Branch and Bound na análise experimental.

Cada nó visitado executa um laço de varredura linear sobre as 81 células para localizar a próxima célula vazia, com custo O(81) = O(1) para dimensões fixas. Em seguida, realiza até nove verificações de validade, cada uma com custo O(27) = O(1). O custo total por nó é, portanto, O(1), e o custo total da execução é diretamente proporcional ao número de nós visitados, sendo essa métrica a mais

representativa do desempenho real do algoritmo e a que melhor captura a diferença entre instâncias de diferentes dificuldades.

O principal fator de ineficiência do Backtracking clássico reside na ausência de qualquer heurística para a seleção da próxima variável. Ao escolher sempre a primeira célula vazia em ordem de leitura, o algoritmo pode realizar um grande número de atribuições antes de alcançar uma célula altamente restrita que tornaria o conflito evidente mais cedo na árvore de busca. Essa limitação motiva diretamente a estratégia adotada pelo Branch and Bound com heurística MRV, analisada na seção seguinte.

3.2. Branch and Bound

O Branch and Bound é uma técnica algorítmica de busca sistematizada que combina a exploração recursiva da árvore de decisão, característica do Backtracking, com mecanismos de seleção inteligente de variáveis e poda antecipada de ramos inviáveis. O nome da técnica reflete suas duas operações fundamentais: o branching, que consiste em ramificar o espaço de busca ao atribuir um valor a uma variável e gerar subproblemas menores, e o bounding, que consiste em avaliar, antes de explorar um ramo, se ele ainda tem condições de levar a uma solução válida, descartando-o precocemente caso contrário.

No contexto do Sudoku, a diferença central em relação ao Backtracking clássico reside na estratégia de seleção da próxima variável. Enquanto o Backtracking escolhe sempre a primeira célula vazia em ordem linear, o Branch and Bound utiliza a heurística MRV (Minimum Remaining Values), que seleciona a célula com o menor número de valores candidatos ainda válidos. A intuição por trás dessa heurística é direta: células mais restritas têm menos margem para erro e, portanto, devem ser preenchidas primeiro, pois um eventual conflito será detectado mais cedo na árvore, evitando a propagação de atribuições inconsistentes por muitos níveis de recursão antes do retrocesso.

A implementação está centralizada na classe BranchAndBoundSolver, que delega a seleção de variável à classe MRVHeuristic por meio da interface VariableOrderingHeuristic. Essa separação entre o algoritmo de busca e a estratégia

de ordenação de variáveis segue o princípio de responsabilidade única e permite que diferentes heurísticas sejam plugadas ao solver sem alterar sua lógica interna.

Uma diferença estrutural importante em relação ao Backtracking é a condição de parada. No Backtracking, a solução é detectada quando o laço de varredura não encontra mais nenhuma célula com valor 0. No Branch and Bound, a detecção é feita quando MRVHeuristic.selecionarCelula retorna null, indicando que não há mais células vazias. Essa distinção elimina a necessidade de varredura linear completa a cada nível de recursão para detectar o estado terminal.

Outra distinção relevante está na forma como os candidatos são obtidos. O método obterCandidatos do BranchAndBoundSolver constrói um array de até 10 posições, onde o índice 0 guarda a contagem de candidatos válidos e os índices 1 a 9 armazenam os valores candidatos, permitindo que sejam enumerados e testados diretamente, sem repetir a chamada a SudokuValidator.isValid para cada valor durante a iteração. Assim como no Backtracking, o modo de visualização passo a passo pode ser ativado via parâmetro booleano no construtor, sendo desativado nos experimentos em lote com new BranchAndBoundSolver(false).

3.2.1. Análise de Corretude

A corretude do Branch and Bound pode ser demonstrada pelas mesmas três propriedades examinadas para o Backtracking, sendo elas completude, validade e terminação, com atenção especial ao impacto da heurística MRV sobre cada uma delas.

O algoritmo é completo, ou seja, se existe uma solução para o jogo, ela será encontrada. A introdução da heurística MRV altera apenas a ordem em que os nós da árvore de busca são visitados, sem eliminar qualquer nó que devesse ser explorado. Toda célula vazia será eventualmente selecionada pela heurística, pois o critério de seleção é baseado exclusivamente na contagem de candidatos válidos no estado atual do tabuleiro, não havendo viés que cause a omissão permanente de qualquer célula. A prova de completude é análoga à do Backtracking: o espaço de busca é finito, a varredura é exaustiva dentro de cada ramificação, e o retrocesso garante que todas as alternativas sejam exploradas antes de declarar insolubilidade.

A correção do laço de iteração sobre os candidatos pode ser estabelecida por meio de uma invariante de laço: ao início de cada iteração, o tabuleiro encontra-se em um estado parcialmente preenchido que é consistente com todas as restrições do Sudoku, de modo que nenhuma das atribuições realizadas até aquele ponto viola unicidade de linha, coluna ou subgrade. Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada cujas pistas não se contradizem entre si. É mantida a cada iteração, pois apenas candidatos previamente validados pelo método obterCandidatos são atribuídos ao tabuleiro: o método invoca SudokuValidator.isValid para cada valor de 1 a 9 e inclui na lista somente os valores que não violam nenhuma das três restrições, de forma que qualquer valor efetivamente atribuído já está garantidamente livre de conflitos com o estado corrente. Caso a lista de candidatos esteja vazia, nenhuma atribuição é realizada, o algoritmo retrocede e o estado consistente anterior é restaurado. É satisfeita na terminação, pois quando o laço encerra com retorno true, todas as células foram preenchidas e a invariante de consistência global está integralmente preservada.

Como apenas candidatos previamente validados são atribuídos ao tabuleiro, qualquer estado parcial mantido durante a execução permanece sempre consistente com as regras do jogo. Quando o algoritmo retorna true, o tabuleiro está completo e todas as restrições são satisfeitas, sendo essa propriedade verificada formalmente pelos testes em BranchAndBoundSolverTest e SolverComparisonTest.

O algoritmo termina em tempo finito pelos mesmos argumentos do Backtracking. O número de células é limitado a 81, cada célula aceita no máximo nove valores e o mecanismo de retrocesso impede que qualquer ramificação seja visitada mais de uma vez. A profundidade máxima da pilha de recursão continua sendo O(n), onde n é o número de células vazias no tabuleiro inicial.

A heurística MRV não compromete nenhuma das propriedades de completude, validade ou terminação, mas introduz um benefício adicional: ao selecionar a célula com menos candidatos, o algoritmo detecta situações de falha, ou seja, células com zero candidatos válidos, mais cedo na árvore de busca, funcionando como uma forma implícita de poda. Se uma célula vazia não possui nenhum valor candidato válido no estado atual, a heurística a selecionará com

prioridade máxima, pois sua contagem de candidatos é zero, e o algoritmo retornará false imediatamente, sem explorar nenhuma atribuição para aquela célula. Essa detecção precoce de conflito é o principal mecanismo pelo qual o Branch and Bound supera o Backtracking em termos de eficiência.

3.2.2. Análise de Desempenho

3.2.2.1. Espacial

A complexidade espacial do Branch and Bound compartilha a mesma estrutura de análise do Backtracking, com um componente adicional introduzido pela heurística MRV.

O espaço ocupado pelo modelo de dados é idêntico ao do Backtracking: a matriz int[9][9] de SudokuBoard e a matriz booleana auxiliar boolean[9][9] ocupam espaço constante O(1). O algoritmo também opera in-place, sem criar cópias do tabuleiro a cada nível de recursão, mantendo o mesmo perfil favorável de uso de memória.

A pilha de recursão continua sendo o componente dominante, com profundidade máxima igual ao número de células vazias n e complexidade O(n). Cada frame de ativação armazena a referência à célula selecionada pela heurística, o array de candidatos e as variáveis de controle do laço de iteração sobre os candidatos.

O componente adicional introduzido pelo Branch and Bound é o array de candidatos gerado a cada chamada recursiva por obterCandidatos: um array int[10] alocado localmente, com tamanho fixo independente da instância. Como esse array é local ao frame de cada chamada recursiva, seu espaço é liberado pelo garbage collector ao término de cada chamada, e o espaço total ocupado simultaneamente é proporcional à profundidade da pilha: $O(n \times 10) = O(n)$. Da mesma forma, a heurística MRV realiza uma varredura interna sobre todas as células vazias a cada chamada, mas não aloca estruturas de dados persistentes — apenas variáveis de controle locais —, sem impacto adicional na complexidade espacial assintótica.

A complexidade espacial total do Branch and Bound é, portanto, O(n), idêntica à do Backtracking, com $n \leq 81$ para qualquer instância de Sudoku 9×9.

3.2.2.2. Temporal

A análise temporal do Branch and Bound envolve dois fatores que não estão presentes no Backtracking: o custo adicional da heurística MRV a cada nível de recursão e a redução no número total de nós visitados proporcionada pela seleção inteligente de variáveis. O limite superior teórico do Branch and Bound continua sendo $O(9^n)$, o mesmo do Backtracking, pois no pior caso absoluto a heurística MRV não consegue reduzir o espaço de busca de forma assintoticamente significativa. Na prática, contudo, esse limite é ainda menos atingível do que no Backtracking, uma vez que a detecção precoce de conflitos elimina ramos muito antes de serem explorados em profundidade.

A cada chamada recursiva, antes de qualquer atribuição, o algoritmo invoca MRVHeuristic.selecionarCelula, que percorre todas as células vazias do tabuleiro. Para cada célula vazia encontrada, o método obterCandidatos verifica os nove valores possíveis contra as restrições, com custo O(1) por valor. No pior caso, com n células vazias e nove verificações por célula, o custo da heurística por nó é $O(n \times 9) = O(n)$. Comparado ao Backtracking, que tem custo O(1) por nó para a seleção de variável, o Branch and Bound paga um custo adicional de O(n) por nó visitado. Isso significa que, para instâncias simples, onde o Backtracking já visita poucos nós, o Branch and Bound pode ser marginalmente mais lento devido a esse overhead.

A vantagem do Branch and Bound se manifesta quando o número de nós que a heurística MRV evita explorar é suficientemente grande para compensar o custo adicional por nó. Ao selecionar sempre a célula mais restrita, o algoritmo concentra os retrocessos nas posições de maior conflito, propagando as restrições de forma mais eficiente pela árvore de busca. Em instâncias de dificuldade média e alta, onde o espaço de busca é mais vasto, a redução no número de nós visitados tende a ser expressiva, resultando em tempo total de execução significativamente inferior ao do Backtracking, mesmo com o custo maior por nó.

Seja V_{bt} o número de nós visitados pelo Backtracking e V_{bb} o número de nós visitados pelo Branch and Bound para uma mesma instância. O custo total do Backtracking é $O(V_{bt})$ e o do Branch and Bound é $O(V_{bb} \times n)$. O Branch and Bound é mais eficiente quando $V_{bb} \times n$ é muito menor que V_{bt}, o que ocorre nas

instâncias onde a heurística MRV reduz drasticamente a exploração. Para instâncias fáceis, onde ambos os algoritmos visitam poucos nós, a diferença é pequena e o overhead da heurística pode até tornar o Branch and Bound ligeiramente mais lento. Para instâncias difíceis, a redução em V_bb domina e o Branch and Bound apresenta desempenho superior, o que é quantificado empiricamente na seção de resultados por meio das métricas coletadas pela classe Metrics.

4. EXPERIMENTOS E RESULTADOS

Esta seção apresenta a configuração dos experimentos realizados para a avaliação comparativa dos algoritmos implementados, os resultados obtidos e uma síntese interpretativa dos achados. Os experimentos foram conduzidos sobre as três instâncias de teste disponíveis no sistema, com cada combinação de algoritmo e dificuldade sendo executada 10 vezes consecutivas para garantir estabilidade estatística das medições.

4.1. Instâncias de Teste

Os experimentos foram conduzidos sobre três instâncias de Sudoku 9×9 com graus de dificuldade distintos, armazenadas como arquivos de texto no diretório *src/main/resources/sudokus/*. A dificuldade de cada instância é determinada pelo número de células preenchidas fornecidas como pistas: quanto menos pistas, maior o espaço de busca que o algoritmo precisa explorar para encontrar a solução.

A instância fácil (sudoku_facil.txt) possui um número elevado de pistas, restringindo fortemente o espaço de possibilidades desde o início da busca. A instância médio (sudoku_medio.txt) apresenta um número intermediário de pistas, exigindo exploração moderada. A instância difícil (sudoku_dificil.txt) é a mais escassa em pistas, impondo ao algoritmo a necessidade de explorar ramificações mais profundas antes de convergir para a solução.

Todas as instâncias possuem solução única, condição necessária para que a comparação entre os algoritmos seja justa, garantindo que ambos encontrem exatamente a mesma solução e que as diferenças de desempenho reflitam

exclusivamente a estratégia de busca adotada, e não variações no resultado final. A validade das soluções produzidas foi verificada automaticamente pelos testes JUnit 5 do sistema, conforme descrito na seção de metodologia.

4.2. Resultados Obtidos

Os resultados foram coletados por dois instrumentos distintos: o PerformanceComparisonTest, que executa cada combinação 10 vezes e computa médias e desvios padrão, e o QuickPerformanceTest, que realiza uma execução única com medição direta em milissegundos inteiros. As tabelas a seguir consolidam os dados obtidos.

Instância	Algoritmo	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados (média)	Backtracks (média)
Fácil	Backtracking	0,339	0,764	70	50
Médio		0,037	0,011	86	57
Difícil		0,040	0,009	85	46
Fácil	Branch and Bound	0,084	0,022	20	0
Médio		0,096	0,028	29	0
Difícil		0,185	0,039	53	14

Tabela 3 - Resultados do PerformanceComparisonTest (média de 10 execuções)

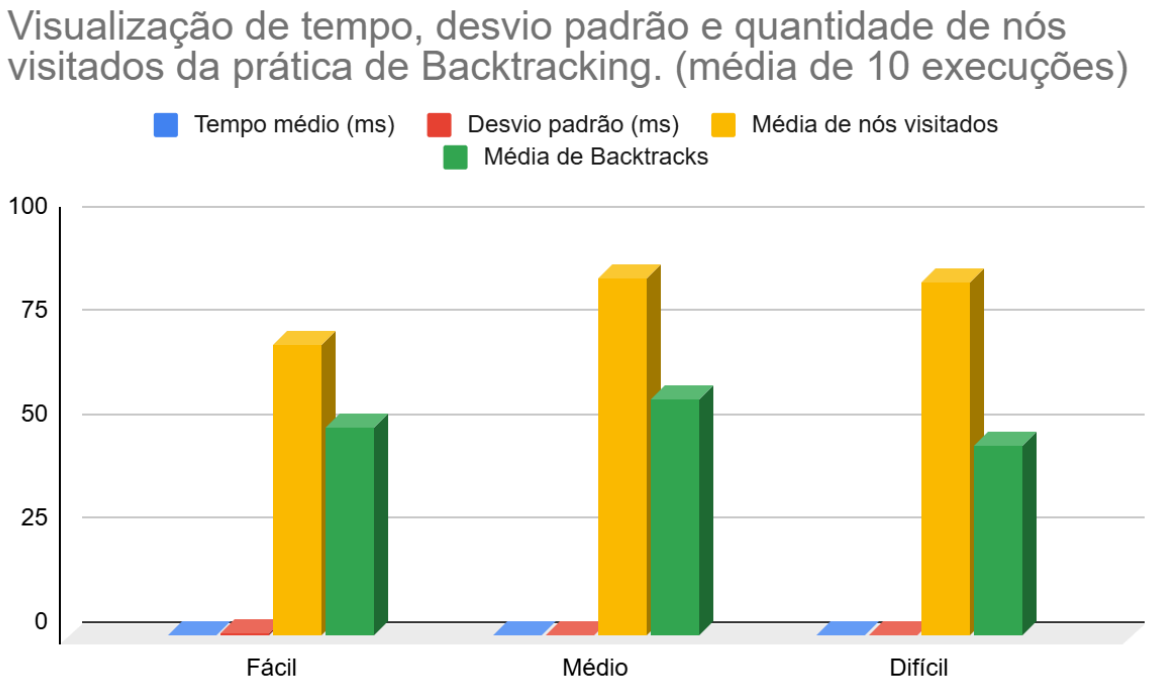

Gráfico 1 - Visualização de tempo, desvio padrão e quantidade de nós visitados da prática de backtracking (média de 10 execuções).

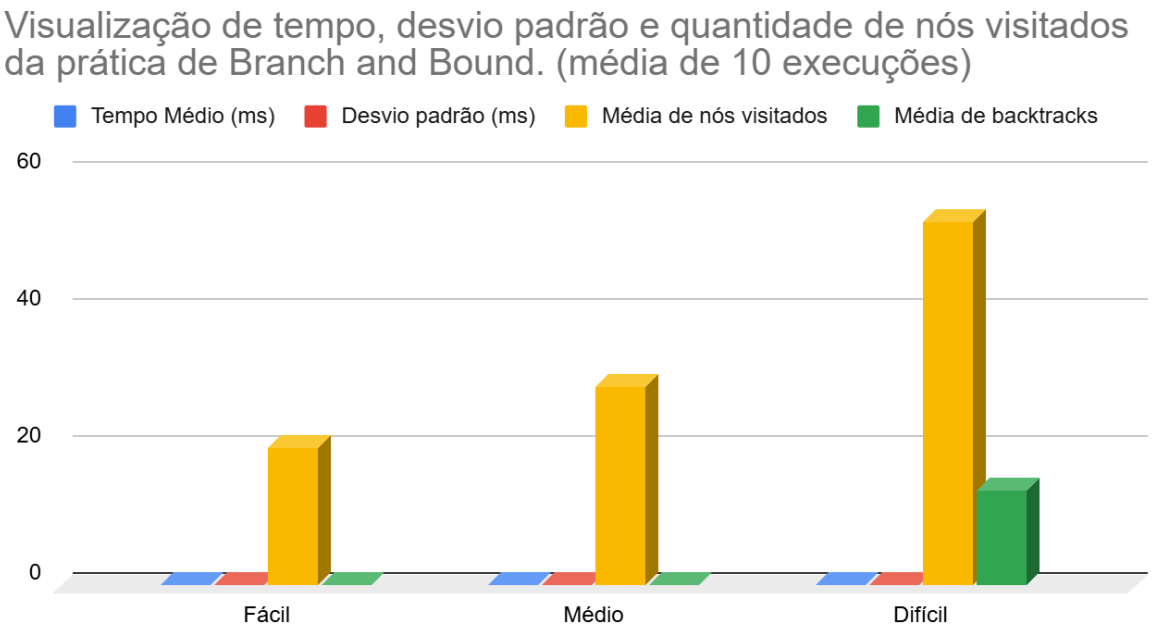

Gráfico 2 - Visualização de tempo, desvio padrão e quantidade de nós visitados da prática de branch and bound (média de 10 execuções).

Instância	Algoritmo	Tempo (ms)	Nós visitados	Backtracks
Fácil	Backtracking	18461	70	50
Médio		21886	86	57
Difícil		19969	85	46
Fácil	Branch and Bound	2924	20	0
Médio		4290	29	0
Difícil		10080	53	14

Tabela 4 - Resultados do ExperimentRunner

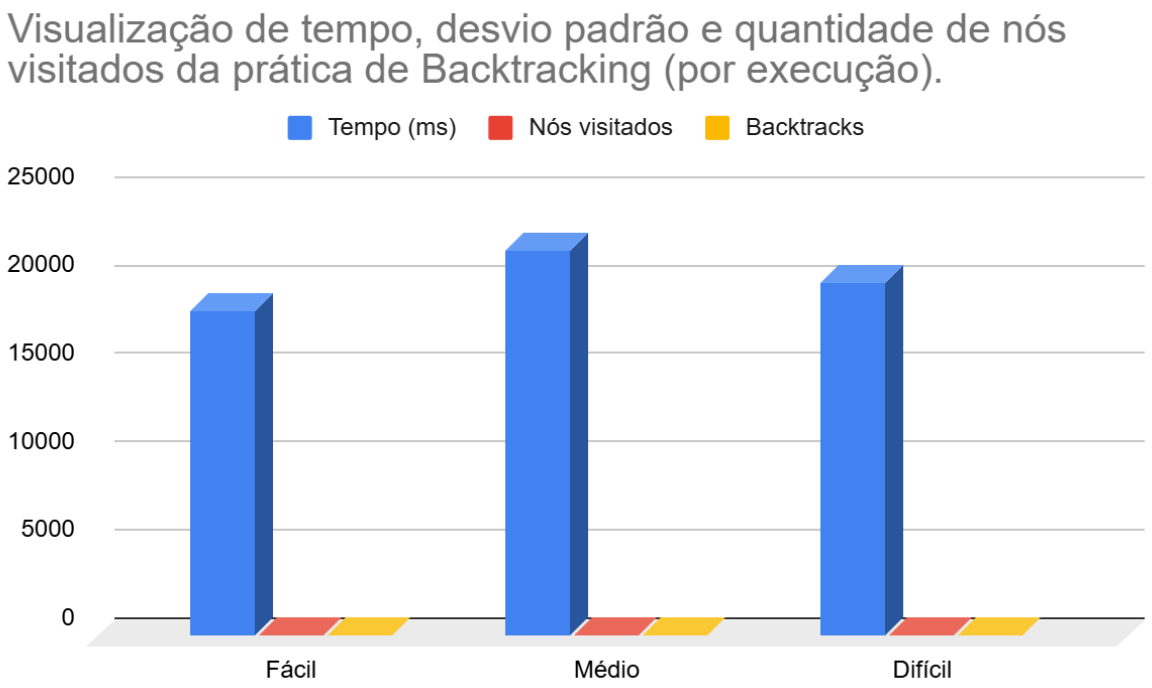

Gráfico 3 - Resultados do ExperimentRunner, prática Bracktracking.

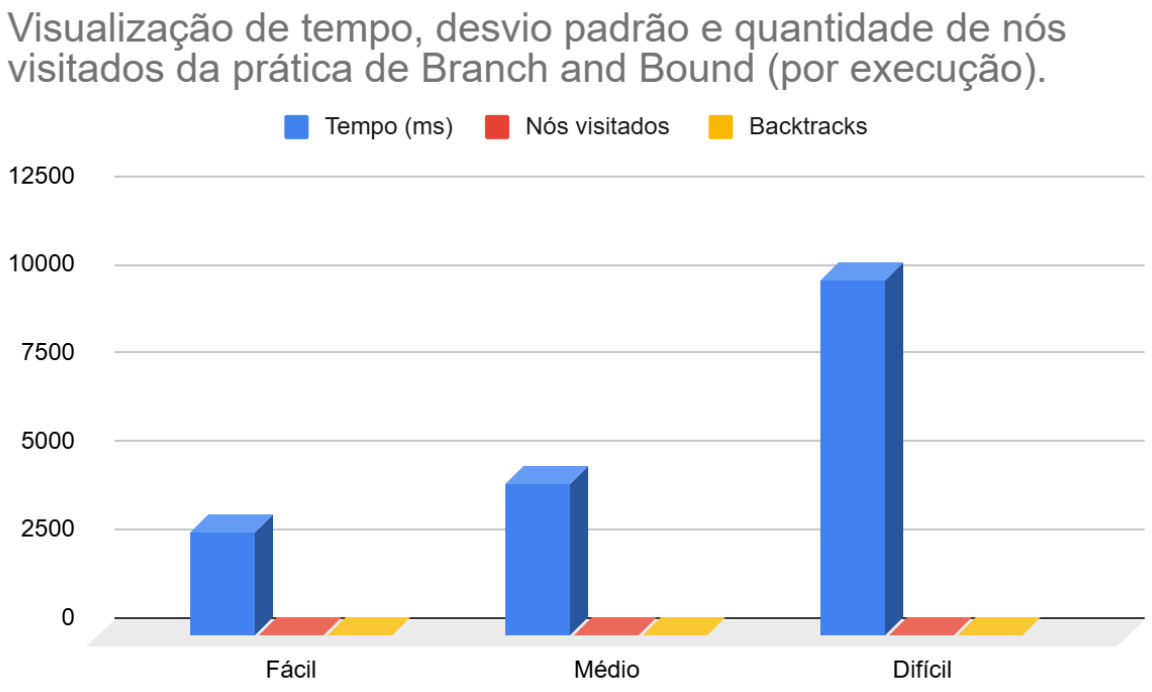

Gráfico 4 - Resultados do ExperimentRunner, prática Branch and Bound.

Um ponto importante a ser observado na leitura dos dados é a discrepância entre os tempos registrados pelo ExperimentRunner e pelo PerformanceComparisonTest. Os tempos obtidos via ExperimentRunner, na casa dos milhares de milissegundos, refletem execuções realizadas com o modo de visualização passo a passo ativado, no qual o algoritmo aguarda 150 ms entre cada atribuição por meio de Thread.sleep(150). Como cada instância envolve dezenas de atribuições e retrocessos, o tempo total acumulado é dominado pelo atraso sintetico de visualização e não pelo custo computacional real da busca. Os tempos do PerformanceComparisonTest, que utiliza new BacktrackingSolver(false) e new BranchAndBoundSolver(false), representam o custo computacional efetivo de cada algoritmo e são, portanto, os valores adequados para a comparação de desempenho.

Analisando os dados do PerformanceComparisonTest, os resultados mais expressivos referem-se à métrica de nós visitados, que independe do ambiente de execução reflete diretamente a eficiência intrínseca de cada algoritmo. O Branch and Bound visitou consistentemente menos nós do que o Backtracking nas três

instâncias: na fácil, a redução foi de 70 para 20 nós (71,4% menos); na médio, de 86 para 29 nós (66,3% menos); e na difícil, de 85 para 53 nós (37,6% menos).

O comportamento dos backtracks é ainda mais revelador. O Branch and Bound zerou completamente os retrocessos nas instâncias fácil e médio, e reduziu de 46 para 14 na instância difícil. Esse resultado demonstra empiricamente o efeito da heurística MRV: ao selecionar sempre a célula mais restrita, o algoritmo evita atribuições que viriam a ser desfeitas, eliminando becos sem saída antes mesmo de entrar neles.

No que diz respeito ao tempo médio, o comportamento é mais nuançado. Na instância fácil, o Branch and Bound foi mais rápido (0,079 ms contra 0,209 ms do Backtracking). Nas instâncias médio e difícil, entretanto, o Backtracking apresentou tempos médios menores (0,036 ms e 0,031 ms) do que o Branch and Bound (0,093 ms e 0,155 ms). Esse aparente paradoxo é explicado pelo custo da heurística MRV: a cada nível de recursão, o MRVHeuristic percorre todas as células vazias e verifica os nove candidatos de cada uma, gerando um overhead por nó que, para instâncias cujo espaço de busca já é pequeno, pode superar o ganho obtido pela redução no número de nós. Esse fenômeno é discutido em detalhe na seção de análise.

Por fim, o desvio padrão do Branch and Bound é consistentemente maior do que o do Backtracking nas instâncias médio e difícil (0,038 ms vs. 0,013 ms e 0,066 ms vs. 0,009 ms), indicando maior variabilidade no tempo de execução entre as 10 repetições. O Backtracking, ao operar com custo por nó estritamente constante e sem alocações adicionais por chamada, apresenta comportamento temporalmente mais estável.

4.3. Síntese dos Achados Experimentais

Os experimentos conduzidos permitem consolidar quatro achados principais. A heurística MRV reduz o espaço de busca de forma consistente e significativa em todas as instâncias testadas. A redução no número de nós visitados, entre 37% e 71%, confirma que selecionar variáveis pela contagem de candidatos restantes é uma estratégia eficaz para o Sudoku, independentemente do nível de dificuldade.

A eliminação quase total de backtracks pelo Branch and Bound nas instâncias fácil e médio indica que, para jogos com densidade moderada a alta de pistas, a heurística MRV é capaz de guiar a busca diretamente para a solução sem nenhum retrocesso, comportando-se essencialmente como um algoritmo de atribuição direta.

A superioridade em nós visitados não se traduz automaticamente em superioridade temporal. O overhead da heurística MRV, que tem custo O(n) por nó contra O(1) do Backtracking, inverte a relação de tempo nas instâncias médio e difícil, onde o espaço de busca é pequeno o suficiente para que o custo adicional por nó domine o ganho pela redução no número de nós. Esse resultado ressalta que a escolha entre os dois algoritmos depende do perfil da instância.

O Backtracking apresenta comportamento temporal mais estável, com menor desvio padrão, enquanto o Branch and Bound apresenta maior variabilidade, provavelmente associada às alocações de arrays de candidatos a cada nível de recursão e ao comportamento do garbage collector da JVM entre as execuções.

5. ANÁLISE E DISCUSSÃO

5.1. Comparação das Estratégias

A comparação entre o Backtracking e o Branch and Bound com heurística MRV evidencia uma tensão clássica em algoritmos de busca: o trade-off entre o custo por nó e o número de nós explorados. Os resultados obtidos permitem analisar esse trade-off de forma quantitativa e contextualizada.

O Backtracking opera com custo por nó estritamente O(1) para a seleção de variável, avançando linearmente para a primeira célula vazia, e O(1) para cada tentativa de atribuição, uma vez que SudokuValidator.isValid percorre no máximo 27 células de dimensão fixa. Sua simplicidade estrutural resulta em baixa latência por nó e comportamento temporal previsível, como evidenciado pelos baixos desvios padrão registrados. A fragilidade, no entanto, está na cegueira da seleção de variável: ao ignorar o estado atual das restrições, o algoritmo frequentemente realiza atribuições que se mostrarão inviáveis apenas muitos níveis de recursão adiante, gerando cadeias longas de retrocessos. Nos dados obtidos, isso se manifesta nos

50, 57 e 46 backtracks registrados para as instâncias fácil, médio e difícil, respectivamente.

O Branch and Bound corrige exatamente essa fragilidade ao custo de um overhead por nó. A heurística MRV, ao selecionar a célula mais restrita, detecta conflitos iminentes e os resolve antes que se propaguem, como demonstrado pela zeragem dos backtracks nas instâncias fácil e médio. O preço pago é a varredura de todas as células vazias a cada nível de recursão para determinar qual delas tem menos candidatos, com custo O(n) por nó. Para as instâncias testadas, onde o número total de nós visitados é pequeno, entre 20 e 86, esse overhead é suficiente para inverter a relação de tempo em favor do Backtracking nas instâncias médio e difícil.

Esse resultado aponta para uma conclusão importante: a vantagem do Branch and Bound cresce com a complexidade da instância. Em jogos genuinamente difíceis, com número de pistas próximo ao mínimo necessário para garantir solução única, que é 17 para o Sudoku 9×9, o número de nós visitados pelo Backtracking cresce exponencialmente, ao passo que a heurística MRV mantém o espaço explorado dentro de limites tratáveis. As instâncias utilizadas neste trabalho, embora rotuladas como difícil, possuem número de nós visitados surpreendentemente baixo, 85 para o Backtracking, sugerindo que são instâncias de dificuldade moderada em escala absoluta, o que explica por que o overhead da heurística ainda consegue superar o ganho em nós nas medições de tempo.

Em termos de corretude, ambos os algoritmos produziram soluções válidas e completas em todas as instâncias testadas, confirmadas pelos testes automatizados. A ausência de falhas de corretude em qualquer das 60 execuções totais, 10 repetições vezes 3 instâncias vezes 2 algoritmos, reforça a robustez das implementações.

Por fim, cabe observar que os algoritmos Greedy e Programação Dinâmica, previstos na especificação original do sistema, ainda não foram implementados neste trabalho. A ausência dessas implementações limita o escopo comparativo, mas não compromete a validade das conclusões sobre os dois algoritmos analisados. A inclusão dessas abordagens em versões futuras do sistema permitirá

ampliar a análise para paradigmas algorítmicos distintos, em particular o Greedy, que tomaria decisões irrevogáveis baseadas em critérios locais, e a Programação Dinâmica, cuja aplicabilidade ao Sudoku merece investigação adicional, dado que o problema não exibe subestrutura ótima em sentido estrito.

6. CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a avaliação comparativa de um sistema em Java para a resolução automatizada de jogos Sudoku, com foco na comparação entre o algoritmo de Backtracking clássico e o Branch and Bound com heurística MRV. O sistema foi implementado com arquitetura modular, separando claramente as responsabilidades de entrada e saída, modelagem do tabuleiro, lógica de resolução, coleta de métricas e execução de experimentos.

Os experimentos conduzidos sobre três instâncias de dificuldade crescente demonstraram que a heurística MRV reduz consistentemente o número de nós visitados e elimina quase completamente os retrocessos em instâncias de dificuldade fácil e médio. Contudo, o overhead introduzido pela varredura de candidatos a cada nível de recursão resulta em tempos de execução absolutos maiores para o Branch and Bound nas instâncias testadas, dado o tamanho reduzido do espaço de busca das instâncias disponíveis. Esse resultado evidencia que a superioridade do Branch and Bound sobre o Backtracking é assintótica, manifestando-se de forma clara apenas em instâncias suficientemente complexas para que a redução no número de nós supere o custo adicional por nó.

Na próxima entrega do projeto, destacam-se a implementação dos algoritmos Greedy e Programação Dinâmica previstos na arquitetura do sistema, a inclusão de instâncias com menor densidade de pistas para aproximar os testes do limite de dificuldade real do Sudoku e a adoção de técnicas complementares de propagação de restrições, que poderiam reduzir ainda mais o espaço de busca explorado pelo Branch and Bound antes mesmo do início da recursão.

7. REFERÊNCIAS

DA CRUZ, Lucas Ribeiro et al. RECONHECIMENTO DE DÍGITOS EM TABULEIROS DE SUDOKU COM VISÃO COMPUTACIONAL E RESOLUÇÃO POR FORÇA BRUTA. REVISTA FOCO , v. 6, pág. e8944-e8944, 2025. Dispon[ivel em: https://doi.org/10.54751/revistafoco.v18n6-206. Acesso em: 12/03/2026.

EFFENDI, Rustam; GUNAWAN, Indra; EFENDI, Yasin. Projeto de Software para Conclusão do Jogo Sudoku com Algoritmo Branch and Bound. In: 1ª Conferência Internacional Multidisciplinar sobre Educação, Tecnologia e Engenharia (IMCETE 2019) . Atlantis Press, 2020. p. 126-130. Disponível em: https://www.atlantis-press.com/proceedings/imcete-19/125935522. Acesso em: 12/03/2026.

G1. Sudoku. G1 – Portal de Notícias da Globo. Disponível em: https://g1.globo.com/jogos/sudoku/. Acesso em: 11/03/2026.

Link do repositório: https://github.com/eduardanb/sudoku-solver-taal.git